Notes on the chapter about queues

The chapter starts from a simple picture: a queue is a line where the first item in is the first item out. What stood out to me is how much of the chapter is about what happens when the line is full or empty, which I had never thought about before.

A bounded queue has a fixed size, and the producer has to decide what to do when it is full. The chapter lists three choices: wait until there is space, throw the new item away, or throw the oldest item away. Each choice suits a different job. Waiting suits work that must not be lost, and dropping the oldest suits a live display where only the latest value matters.

The part I liked best was the comparison of an array-based queue with a linked one. Removing from the front of a plain array is slow because every other item shifts along. The chapter fixes this with a circular buffer, where two indexes chase each other around a fixed array and nothing moves. I drew it on paper twice before the wrap-around made sense.

A short section on priority queues surprised me. A heap lets you take out the most important item first without sorting everything, and the chapter says this is how many schedulers work.

Questions I still have: how does a circular buffer know it is full, as opposed to empty, when the two indexes are equal in both cases? The chapter mentions keeping a count, but also a trick with one unused slot, and I would like to understand when to prefer which.